

Observability-Driven Intelligence for Anomaly Detection in Cloud-Native Distributed Microservices

Ngoc Duc Anh Nguyen ✉

School of Built Environment, Engineering and Computing, Leeds Beckett University, Leeds, UK

Satish Kumar ✉

School of Built Environment, Engineering and Computing, Leeds Beckett University, Leeds, UK

Nawar Jawad ✉

School of Built Environment, Engineering and Computing, Leeds Beckett University, Leeds, UK

Abstract

Automated fault diagnosis in cloud-native microservice architectures remains an open challenge: the three pillars of observability—metrics, logs, and distributed traces—each capture a distinct failure dimension, yet most production AIOps deployments exploit only one signal modality, leaving cross-modal causal chains undetected until user-visible degradation occurs. We present an observability-driven AIOps platform that closes this gap through continuous, fully offline multi-modal intelligence, motivated by AML compliance environments where data residency regulations categorically prohibit telemetry export to third-party infrastructure. Eight Kubernetes-native AIOps services ingest a three-pillar telemetry pipeline (Prometheus, Loki, Grafana Tempo) and apply a three-algorithm ML pipeline in sequence: Isolation Forest anomaly scoring over 60-second stream-windowed feature vectors, Pearson correlation root cause ranking over sliding observational windows, and ordinary least-squares regression on p95 latency trends for predictive SLO breach forecasting. Rather than delegating incident narration to a cloud-hosted model, the platform synthesises MELT context through a locally deployed Qwen2.5-3B large language model [27] served via Ollama [24] and continuously fine-tuned with LoRA adapters [16] on resolved incident outcomes—producing structured, actionable JSON diagnostic reports with no external API call leaving the cluster. The platform is evaluated through controlled Chaos Mesh [5] fault injection across four fault classes, addressing three research questions: whether fusing all three signal modalities outperforms any single-modal configuration (RQ1), whether correlation-based root cause ranking localises faults more accurately than threshold or trace-error baselines (RQ2), and what overhead full-fidelity distributed tracing imposes on regulated workloads (RQ3). Preliminary validation confirms fault detection within two 60-second analysis cycles, on-cluster LLM narration at 3–5 seconds per report, and zero telemetry egress over a 2-hour injection session—establishing the platform as both a privacy-preserving operational tool and a reproducible experimental substrate for quantitative AIOps evaluation.

2012 ACM Subject Classification Computer systems organization → Cloud computing; Computing methodologies → Anomaly detection; Information systems → Data management systems

Keywords and phrases anomaly detection, microservices, observability, machine learning, Kubernetes, distributed tracing, root cause analysis, AIOps, anti-money laundering

Digital Object Identifier 10.4230/LIPIcs...

Funding This research is supported by Leeds Beckett University.

1 Introduction

Distributed microservice architectures now coordinate hundreds of containerised services across Kubernetes clusters, each emitting metrics, logs, and distributed traces at sub-second resolution [4]. Anomalies in such systems manifest as subtle cross-service causal chains—a



© Ngoc Duc Anh Nguyen, Satish Kumar, and Nawar Jawad;
licensed under Creative Commons License CC-BY 4.0

Leibniz International Proceedings in Informatics

LIPICS Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

memory leak first inflects a metric, propagates to trace latency shifts, and eventually cascades into error-log floods—patterns invisible to any single-modal detector and intractable for human operators at the telemetry volumes modern clusters produce.

In regulated domains such as anti-money laundering (AML), this challenge is compounded by data residency obligations. Financial regulations in most jurisdictions explicitly prohibit exporting operational telemetry to third-party cloud services [18], categorically excluding commercially hosted AIOps platforms precisely in the environments where automated anomaly detection is most consequential. AML workloads further exhibit architectural heterogeneity—synchronous KYC REST APIs, asynchronous Kafka event streams, and stateful case-management workflows with legally mandated lifecycle invariants—that generates telemetry across fundamentally different operational patterns simultaneously.

Existing approaches are inadequate on two fronts. Rule-based monitors evaluate individual metrics against static thresholds, generating excessive false-positive alert volumes and failing on distributed cross-service anomalies [31]. Supervised learning approaches require labelled training data unavailable for novel failure modes. Commercial AIOps platforms—Dynatrace Davis AI [8] and Datadog Watchdog [7]—apply sophisticated correlation algorithms but mandate continuous telemetry export to vendor-managed infrastructure. No existing open-source platform unifies multi-modal anomaly detection, correlation-based RCA, and predictive SLO forecasting as a privacy-preserving, fully offline Kubernetes-native service.

We address this gap with two complementary design decisions: (i) all intelligence executes within the cluster—Isolation Forest [21] anomaly scoring, Pearson correlation root cause ranking, OLS SLO breach prediction, and Qwen2.5-3B [27] incident narration fine-tuned with LoRA [16] via Ollama [24], with no external API access; and (ii) the AML testbed uses a transactional outbox pattern that dual-routes domain events as both Kafka streams and OpenTelemetry span events carrying business attributes, providing naturally labelled ground-truth anomaly signals without secondary annotation.

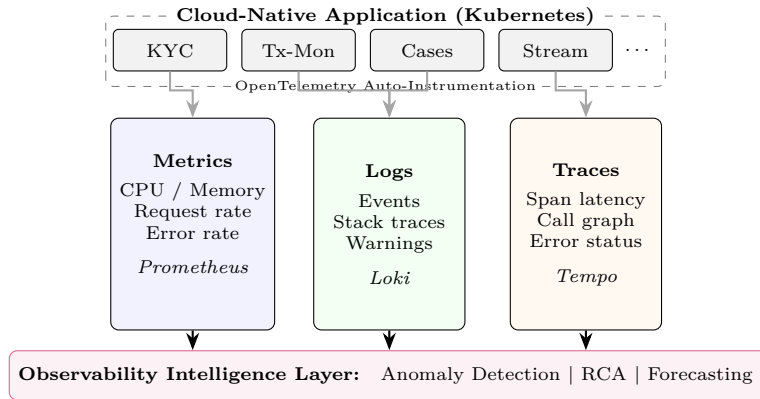
The main contributions are: (i) a production-grade, open-source eleven-service Kubernetes platform for AML-domain observability with a fully offline intelligence layer satisfying financial data residency requirements; (ii) a domain event-based instrumentation strategy deriving ground-truth anomaly labels from AML compliance state transitions via OpenTelemetry span attributes; (iii) a three-algorithm offline ML pipeline (Isolation Forest, Pearson correlation RCA, OLS SLO prediction) integrated with a locally fine-tuned Qwen2.5-3B served via Ollama; and (iv) a controlled Chaos Mesh fault injection evaluation protocol addressing signal sufficiency (RQ1), ML-driven RCA localisation (RQ2), and distributed tracing overhead on regulated workloads (RQ3).

2 Background and Related Works

2.1 Cloud-Native Observability

Observability in distributed systems is the capacity to reconstruct application state from emitted telemetry without requiring instrumentation changes or direct access to production processes [1]. This distinguishes observability from monitoring: monitoring tests predefined conditions, whereas observability enables answering questions not anticipated at deployment time.

The operational realisation of observability rests on three interdependent signal types (Fig. 1). *Metrics* are scalar measurements sampled at regular intervals—CPU utilisation, request throughput, error rates—providing low-overhead, high-frequency health indicators; Prometheus is the de facto standard for Kubernetes metrics collection [2]. *Logs* are immutable



■ **Figure 1** Three-pillar cloud-native observability. Microservices auto-instrumented via OpenTelemetry emit metrics (Prometheus), logs (Loki), and distributed traces (Tempo). The intelligence layer fuses all three signal types for anomaly detection, RCA, and predictive forecasting.

timestamped records of discrete events; Loki extends the Prometheus model to log aggregation via LogQL [11]. *Distributed traces* follow individual requests across services, recording span durations, parent-child call relationships, and propagated context—providing the causal structure that neither metrics nor logs alone supply; Grafana Tempo provides a cost-effective trace backend for Kubernetes-scale ingestion [12].

The observability challenge is amplified in cloud-native architectures where cascading failures—a transient spike in one shared layer propagating through dozens of dependent services [31]—and ephemeral pod lifetimes make historical baselines a moving target. Sigelman et al. first documented this diagnostic challenge at Google scale with Dapper [30]; OpenTelemetry has since unified instrumentation APIs across languages and vendors [25]. The AIOps vision [6]—applying ML to live operational telemetry to replace manual incident triage—is further constrained in AML environments by data residency requirements that force all intelligence to execute on-premises.

2.2 Anomaly Detection in Distributed Microservices

An anomaly is any observation deviating meaningfully from expected operational behaviour, where expectation is established from historical baselines or learned statistical models [31]. In microservices, anomalies manifest across modalities with time lag: a memory leak first appears as a subtle GC-pause metric anomaly, then as elevated downstream trace latency, and finally as timeout error logs—the separation between these manifestations defines the diagnostic difficulty of distributed systems.

At the *metric level*, anomalies include CPU throttling, memory pressure, and distributional latency shifts. At the *trace level*, they appear as unusual span durations, unexpected call paths, or elevated error rates visible only via the full request dependency graph [20]. At the *log level*, anomalies surface as unseen event templates, anomalous repetition rates, or unexpected sequences [15]. Because no single signal modality captures all three dimensions simultaneously, effective detection in microservices must be inherently multi-modal—correlating all signals within a unified temporal context and applying algorithm-specific detectors per modality.

■ **Table 1** Capability comparison of AIOps platforms. ✓ = supported, × = not supported.

System	Multi-modal	Offline	RCA	Local LLM	AML-ready
Dynatrace Davis AI [8]	✓	×	✓	×	×
Datadog Watchdog [7]	✓	×	✓	×	×
LogGPT [26]	×	×	×	×	×
Han et al. [14]	✓	✓	✓	×	×
Ours	✓	✓	✓	✓	✓

2.3 Related Works

AIOps research has produced specialised solutions per modality. For metrics, Liu et al. [21] established Isolation Forest as an efficient anomaly scorer for high-dimensional metric matrices; Malhotra et al. [23] used LSTM Autoencoders for multivariate time-series reconstruction. For traces, Liu et al. [20] demonstrated unsupervised deep Bayesian networks for latency anomaly localisation without labelled data. For logs, He et al. [15] showed semantic embedding with sequence detectors achieves high recall on unseen anomalies; LogGPT [26] reasons over log semantics without templates at the cost of inference latency and external API dependency. For multi-modal RCA, Han et al. [14] proposed holistic causal graph correlation across all three pillars; Lomio et al. [22] confirmed that multi-modal ensemble approaches consistently outperform single-algorithm solutions.

Table 1 positions this work against the state of the art. No existing system combines all five properties—multi-modal signals, fully offline operation, correlation-based RCA, local LLM narration, and AML-domain readiness—in a single open-source Kubernetes-native platform.

The research gap addressed here is threefold: no published open-source platform provides multi-modal detection + RCA + SLO forecasting as a unified offline Kubernetes-native service; existing offline ML systems do not integrate fine-tunable local LLMs for incident narration; and prior evaluations rely on synthetic benchmarks (DeathStarBench, TrainTicket) lacking domain-specific semantic richness—the AML testbed introduced here provides naturally labelled anomalies from compliance domain state machines.

3 Proposed Approach

The AMLOps-AIOps platform comprises eleven Kubernetes-hosted microservices in two functional layers: an AML domain testbed generating semantically labelled financial compliance telemetry, and a dedicated AIOps analysis layer continuously monitoring and diagnosing that telemetry. All intelligence executes within the cluster with no external API dependency.

3.1 System Architecture

Fig. 2 shows the complete platform topology deployed across three namespaces: **aml** (AML domain services), **aiops** (analysis layer), and **monitoring** (observability stack).

AML Domain Layer. Three Spring Boot [33] 3.x microservices implement the testbed: *customer-kyc* performs KYC identity verification and risk scoring; *transaction-monitoring* evaluates transactions against a four-rule AML engine; *case-management* manages investigation lifecycles from alert through regulatory reporting. All three follow a hexagonal DDD [9] architecture with OpenTelemetry SDK auto-instrumentation and a transactional

■ **Table 2** Kafka topic families and their consumers.

Topic Family	Producer	Consumer
<code>aml.{cust,txn,cases}.events</code>	AML services	Telemetry Collector
<code>aml.llm.analysis</code>	LLM Engine	Decision Engine
<code>aiops.telemetry.</code> <code>{metrics,logs,traces}</code>	AML svcs / Prometheus	LLM Engine
<code>aiops.features</code>	Stream Processor	ML Engine
<code>aiops.incidents</code>	ML Engine	LLM / Decision Engines
<code>aiops.decisions</code>	Decision Engine	Remediation Engine
<code>aiops.actions</code>	Remediation Engine	Feedback Service
<code>aiops.outcomes</code>	Feedback Service	LLM Engine (LoRA)
<code>aiops.service.heartbeat</code>	All AIOps services	Telemetry Collector

outbox pattern [29] guaranteeing exactly-once domain event delivery to Kafka [19]. Domain events are simultaneously emitted as OpenTelemetry span events carrying business attributes (`risk.band`, `case.id`, `customer.id`), making ground-truth anomaly labels a natural byproduct of compliance operation rather than secondary annotation.

Message Bus. Apache Kafka (KRaft mode, single broker) carries ten topic families (Table 2): AML domain events, AML LLM analysis, AIOps telemetry signals (metrics/logs/traces), feature windows, detected incidents, remediation decisions, executed actions, resolved outcomes, and 30-second service heartbeats.

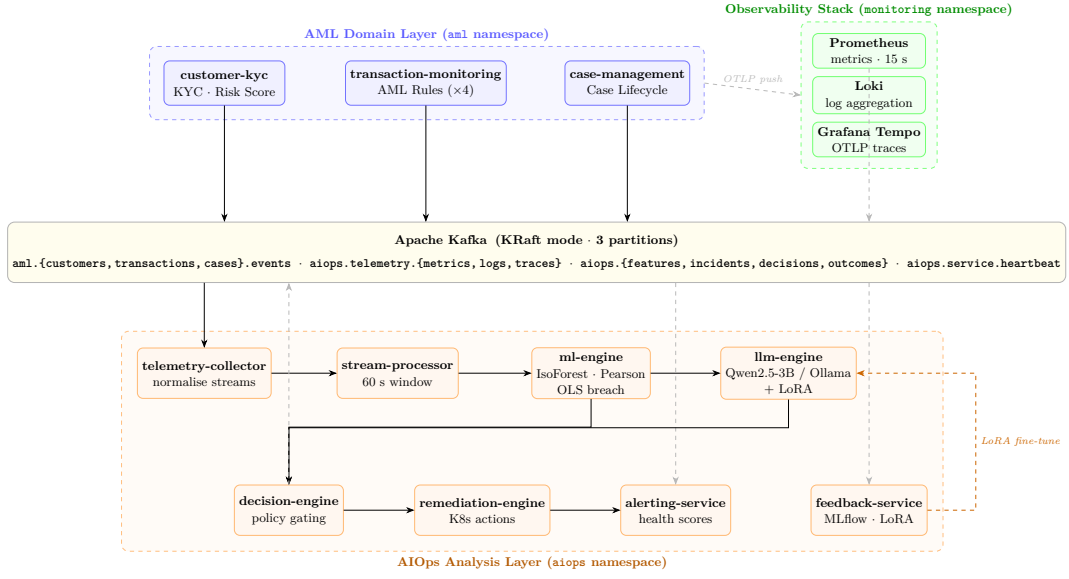
Observability Stack. The `monitoring` namespace hosts Prometheus (15-second metric scraping), Loki (log aggregation via pod label streams), and Grafana Tempo (distributed trace collection via OTLP push) [2, 11, 12]. All AML services emit traces at 100% sampling by default; the `TRACE_SAMPLE_RATE` environment variable switches between full, sampled, and disabled tracing without redeployment to support RQ3 overhead experiments.

Network Perimeter. Kubernetes NetworkPolicy manifests enforce namespace isolation: AML services hold egress only to Kafka broker ports and the `monitoring` OTLP endpoints; AIOps services additionally hold read access to the Prometheus and Loki HTTP APIs; only the Remediation Engine holds write access to the Kubernetes API Server, scoped to pod-restart and HPA-patch operations within the `aml` namespace. This least-privilege perimeter implements the AML data-residency boundary at the network layer, ensuring no telemetry can be routed to external systems regardless of application-level configuration.

3.2 Problem Formulation

Microservices Model. Let $\mathcal{S} = \{s_1, s_2, \dots, s_N\}$ be the set of N microservices ($N=11$ in this platform). Service interactions define a directed dependency graph $\mathcal{G} = (\mathcal{S}, \mathcal{E})$, where $(s_i, s_j) \in \mathcal{E}$ exists if s_i invokes s_j via HTTP/gRPC or produces to a Kafka topic consumed by s_j . In the AML testbed the primary synchronous chain is *customer-kyc* $\rightarrow$ *transaction-monitoring* $\rightarrow$ *case-management*; asynchronous edges connect each AML service to the AIOps pipeline through domain-event Kafka topics.

Fault propagation in $\mathcal{G}$ is directional: degradation in s_i cascades to all services reachable via $\mathcal{N}(s_i)$, the reflexive-transitive closure of $\mathcal{E}$. The Root Cause Ranking algorithm (Problem 2) restricts Pearson correlation search to pairs (s_j, m_k) where $s_j \in \mathcal{N}(s_{\text{anom}})$, reducing the candidate space from $O(N \cdot K)$ to $O(|\mathcal{N}(s_{\text{anom}})| \cdot K)$, where K is the number of distinct metric names. The topology is reconstructed continuously from trace parent-child span relationships



■ **Figure 2** AMLOps-AIOps platform architecture. AML services (blue) publish compliance events to Kafka and push telemetry to the observability stack (green). The AIOps pipeline (orange): Telemetry Collector → Stream Processor → ML Engine → LLM Engine → Decision Engine → Remediation Engine. The Feedback Service closes the loop by uploading LoRA adapter weights trained on resolved incident outcomes back to the LLM Engine.

185 via W3C TraceContext [34] headers and from Kafka consumer-group registrations observed
186 by the Telemetry Collector.

187 Each service s_i emits a multi-modal telemetry context at analysis cycle t : $\mathcal{T}^{(t)} =$
188 $\langle \mathbf{M}^{(t)}, \mathbf{L}^{(t)}, \mathbf{D}^{(t)} \rangle$, where $\mathbf{M}^{(t)} \in \mathbb{R}^{P \times \Delta}$ is the matrix of P Prometheus metric time series
189 over window Δ ; $\mathbf{L}^{(t)} = \{l_j\}_{j=1}^Q$ is the set of Q log entries within $[t-\Delta, t]$; and $\mathbf{D}^{(t)} = \{d_k\}_{k=1}^R$
190 is the set of R trace spans as $\langle \text{service}, \text{duration}, \text{status}, \text{parent_id} \rangle$ tuples.

191 An *operational anomaly* at cycle t is:

$$192 \quad \text{anomaly}(s_i, t) \Leftrightarrow P_{\mathcal{P}_{\text{normal}}}(\varphi_i^{(t)}) < \epsilon \quad (1)$$

193 Since $\mathcal{P}_{\text{normal}}$ is non-stationary and high-dimensional, Isolation Forest provides an efficient
194 surrogate without parametric distributional assumptions. The Stream Processor computes a
195 four-dimensional feature vector per (service, metric) pair over 60-second tumbling windows:

$$196 \quad \varphi_{ij}^{(t)} = [\mu_{ij}, x_{ij}^{\max}, \dot{x}_{ij}, n_{ij}]^T \in \mathbb{R}^4 \quad (2)$$

197 where μ_{ij} is the window mean, $x_{ij}^{\max}$ the window maximum, $\dot{x}_{ij}$ the rate of change, and n_{ij}
198 the sample count.

199 **Problem 1 (Anomaly Detection).** Given $\varphi^{(t)} \in \mathbb{R}^4$, produce an anomaly score $s^{(t)} \in [0, 1]$
200 from an Isolation Forest [21] decision function $d(\cdot)$ (contamination $\gamma=0.05$):

$$201 \quad s^{(t)} = \max\left(0, \min\left(1, 0.5 - d\left(\varphi^{(t)}\right)\right)\right) \quad (3)$$

202 A binary flag is raised when $s^{(t)} > \theta_{\text{anom}}$ ($\theta_{\text{anom}}=0.5$). The model bootstraps on 500 synthetic
203 samples and refits on 50 new labelled samples from operator feedback.

204 **Problem 2 (Root Cause Ranking).** When an anomaly is detected in s_i , rank all (service,
205 metric) pairs by Pearson correlation with the anomalous reference trajectory $\mathbf{x}_{\text{ref}} \in \mathbb{R}^W$

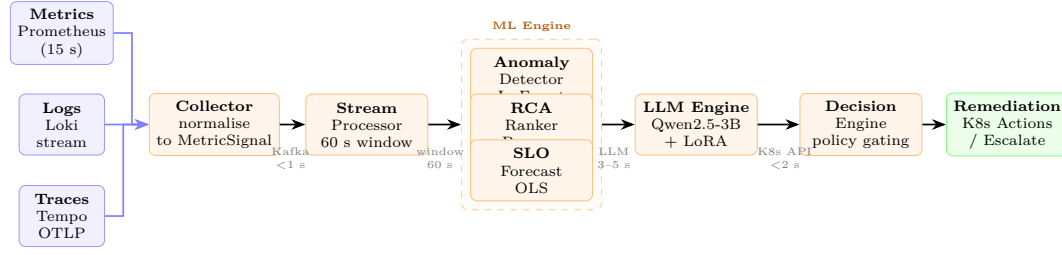


Figure 3 End-to-end detection pipeline. Raw multi-modal telemetry is normalised by the Telemetry Collector, aggregated into 60-second feature windows by the Stream Processor, and analysed by the three-algorithm ML Engine in parallel (anomaly scoring, RCA ranking, SLO breach prediction). The LLM Engine synthesises MELT context into a structured report, which drives policy-gated remediation. Total worst-case detection-to-action latency is $\approx 83\text{--}85\text{ s}$.

($W=100$):

$$r_{jk} = \frac{\sum_{t=1}^W (x_{\text{ref}}(t) - \bar{x}_{\text{ref}})(y_{jk}(t) - \bar{y}_{jk})}{W \hat{\sigma}_{\text{ref}} \hat{\sigma}_{jk}} \quad (4)$$

The top-10 pairs ranked by $|r_{jk}|$ form the root cause ranking.

Problem 3 (SLO Breach Prediction). Fit OLS regression on p95 response latency over $W_b=10$ observations:

$$\hat{p}_{95}(t) = \beta_0 + \beta_1 t, \quad \hat{T}_{\text{breach}} = \frac{\theta_{\text{SLO}} - \hat{\beta}_0}{\hat{\beta}_1} \quad (5)$$

where $\theta_{\text{SLO}} = 500\text{ ms}$. If $\hat{\beta}_1 \leq 0$, no breach is predicted ($\hat{T}_{\text{breach}} \leftarrow 999$).

Problem 4 (LLM Incident Narration). Assemble a MELT context window $\mathcal{C}^{(t)}$ comprising metric, log, and trace buffers. Given the current incident $\mathcal{I}^{(t)}$:

$$\mathcal{R} = f_{\text{LoRA}}(\mathcal{P}(\mathcal{I}^{(t)}, \mathcal{C}^{(t)})) \quad (6)$$

where f_{LoRA} is Qwen2.5-3B [27] fine-tuned via PEFT LoRA [16, 17] and served through Ollama [24]. Output $\mathcal{R}$ conforms to the JSON schema `{explanation, rootCause, recommendation, amlRisk, confidence}`.

Problem 5 (Decision Policy). The Decision Engine selects a remediation action via a piecewise policy on confidence c , blast radius $B \in \{\text{LOW}, \text{MEDIUM}, \text{HIGH}\}$, and ETA $\hat{T}_{\text{breach}}$:

$$\pi(c, B, \hat{T}) = \begin{cases} \text{SCALE-OUT}, & c > 0.90, B=\text{LOW}, \hat{T} < 5 \\ \text{RESTART-POD}, & c > 0.90, B=\text{LOW}, \hat{T} \geq 5 \\ \text{SCALE-OUT}, & c > 0.75, B=\text{MEDIUM} \\ \text{ESCALATE}, & \text{otherwise} \end{cases} \quad (7)$$

Problem 6 (Service Health Score). The Alerting Service maintains $h_i(t) \in [0.05, 1]$ aggregating pod liveness, JVM CPU utilisation, and process-level CPU rate:

$$h_i(t) = \max(0.05 \cdot u_i(t), c_i^{\text{JVM}}(t), \hat{c}_i^{\text{sys}}(t)) \quad (8)$$

■ **Table 3** AIOps analysis layer service summary.

Service	Stack	Function	Algorithm / Tool
Telemetry Collector	Spring Boot	Stream normalisation	MetricSignal canonical format
Stream Processor	Spring Boot / Kafka	Feature extraction	60 s tumbling window (Eq. 2)
ML Engine	Python FastAPI	Three-algorithm pipeline	IsoForest, Pearson, OLS
LLM Engine	Python FastAPI	Incident narration	Qwen2.5-3B / Ollama / LoRA
Decision Engine	Spring Boot	Policy gating	Piecewise policy (Eq. 7)
Remediation Engine	Spring Boot	Kubernetes actions	Pod restart, HPA patch
Alerting Service	Spring Boot	Health scoring	Multi-source score (Eq. 8)
Feedback Service	Spring Boot	Continuous learning	MLflow / PEFT LoRA

where $u_i(t) \in \{0, 1\}$ is the Kubernetes `up` indicator. The floor $0.05 \cdot u_i(t)$ distinguishes running-but-idle pods from crashed pods ($u_i=0 \Rightarrow h_i=0$). A service is classified as *actively degraded* if $h_i(t) > \mu_{h_i} + 2\sigma_{h_i}$ over $W_h=20$ preceding observations; the count of degraded services determines blast radius B .

3.3 Architecture Components

Table 3 summarises all eight AIOps analysis services. The subsections below elaborate the four services whose implementations contain algorithmic or integration novelty not fully captured by the table.

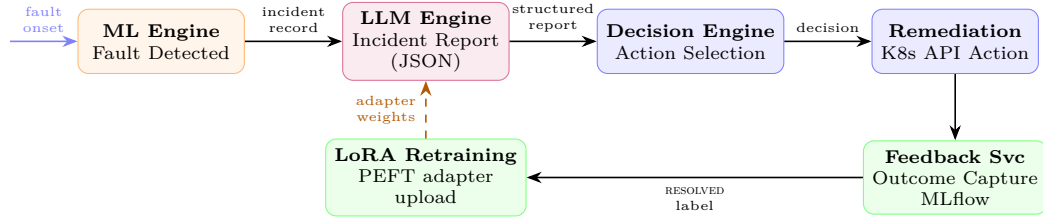
AML Domain Services. The *transaction-monitoring* service evaluates each transaction against four AML rules (AML-101 high-value, AML-202 velocity, AML-303 structuring, AML-404 high-risk corridor). Rule scores $c_r \in [0, 100]$ are combined with a diminishing-returns formula:

$$R = 100 \cdot \left(1 - \prod_{r \in \mathcal{R}_{\text{fired}}} \left(1 - \frac{c_r}{100} \right) \right) \quad (9)$$

Case state transitions (OPEN $\rightarrow$ UNDER-INVESTIGATION $\rightarrow$ ESCALATED $\rightarrow$ PENDING-REVIEW $\rightarrow$ CLOSED) are emitted as OpenTelemetry span events that simultaneously label the corresponding metric window as anomalous.

Closed-Loop Continuous Learning. Fig. 4 illustrates the feedback architecture. The Feedback Service captures SLO outcomes before and after each remediation action (via Prometheus queries), labels each as RESOLVED, NO_EFFECT, or DEGRADED_FURTHER, logs to MLflow [35], and publishes records to `aiops.outcomes` for LoRA fine-tuning of the LLM Engine via the PEFT library [17]. The Decision Engine applies the piecewise policy (Eq. 7), where blast radius is computed as: one degraded service $\rightarrow$ LOW; two $\rightarrow$ MEDIUM; three or more $\rightarrow$ HIGH.

ML Engine. The ML Engine (Python FastAPI [28]) hosts three algorithms in sequence. (i) *Anomaly Detection*: Isolation Forest (contamination $\gamma=0.05$, Eq. 3) is trained online; 500 synthetic bootstrap samples initialise the model at startup, and the model refits when 50 new operator-labelled samples accumulate from the Feedback Service. (ii) *Root Cause Ranking*: Pearson correlation over per-(service, metric) sliding buffers of $W=100$ observations (Eq. 4), returning the top-10 most correlated (service, metric) pairs as the root cause ranking.



■ **Figure 4** Closed-loop continuous learning. Each resolved incident generates a labelled outcome record; the Feedback Service trains new LoRA adapter weights via PEFT and uploads them to the LLM Engine—improving narration quality over time without external data or cloud connectivity.

(iii) *SLO Breach Prediction*: OLS regression on the last $W_b=10$ p95 latency values sampled at 30-second intervals (Eq. 5), projecting time-to-SLO-violation in minutes. Detected incidents are serialised as JSON and published to `aiops.incidents` for downstream consumption by the LLM Engine and Decision Engine.

LLM Engine. The LLM Engine (Python FastAPI) implements Eq. 6. It maintains three MELT buffers: `metric_buffer` (sliding dict of recent feature records), `log_buffer` (deque of the 10 most recent Loki entries consumed from `aiops.telemetry.logs`), and `trace_buffer` (deque of the 5 most recent spans from `aiops.telemetry.traces`). On consuming an incident from `aiops.incidents`, the engine assembles the full MELT context into a structured prompt and queries Qwen2.5-3B [27] via the Ollama REST API (<http://ollama:11434>). LoRA adapter weights [16] trained via PEFT [17] on resolved incident outcomes from `aiops.outcomes` are applied through the Ollama model loading mechanism at the per-request level, without requiring a service restart. The output JSON report is published to `aml.llm.analysis`.

Decision Engine and Remediation Engine. The Decision Engine (Spring Boot) consumes both `aiops.incidents` and `aml.llm.analysis`, applies the piecewise policy (Eq. 7), and publishes `RemediationDecision` records to `aiops.decisions`. Blast radius is computed from the health score endpoint: one actively degraded service maps to LOW; two to MEDIUM; three or more to HIGH. The Remediation Engine subscribes to `aiops.decisions` and executes Kubernetes API actions—pod restarts and HPA target adjustments—while publishing execution outcomes to `aiops.actions`. Actions are executed autonomously when the `auto-execute` ConfigMap flag is set to true; otherwise, an ESCALATE event is published to `aiops.incidents` for human review via the Alerting Service REST endpoint.

Feedback Service and Continuous Learning. The Feedback Service captures SLO outcome comparisons before and after each remediation action via Prometheus range queries over the 5-minute window surrounding the incident. Each outcome is labelled as RESOLVED (p95 latency returned below θ_{SLO}), NO_EFFECT (p95 unchanged), or DEGRADED_FURTHER (p95 increased). Labelled outcomes are logged to MLflow [35] for experiment tracking and published to `aiops.outcomes`. When 50 new RESOLVED or NO_EFFECT outcomes accumulate, the PEFT library [17] initiates a LoRA fine-tuning job that updates the adapter weights held on the cluster PersistentVolume, which the LLM Engine loads at next request. This closed-loop architecture (Fig. 4) enables incident narration quality to improve continuously from operational experience without any external data or cloud connectivity.

Platform Latency Budget. All tunable parameters (γ , W , W_b , θ_{SLO} , θ_{anom} , `auto-execute`) are exposed as Kubernetes ConfigMap entries, enabling live adjustment without service restart. The end-to-end detection-to-action latency decomposes as: Prometheus scrape lag (≤ 15 s) + Kafka propagation (< 1 s) + stream-window accumulation (60 s) + ML inference

■ **Table 4** AIOps ConfigMap parameters and default values.

Parameter	Default	Effect
CONTAMINATION (γ)	0.05	IsoForest expected anomaly fraction
CORRELATION_WINDOW (W)	100	RCA sliding window (observations)
BREACH_WINDOW (W_b)	10	SLO OLS regression window (observations)
SLO_THRESHOLD_MS (θ_{SLO})	500	p95 latency SLO (milliseconds)
ANOMALY_THRESHOLD (θ_{anom})	0.5	IsoForest score detection threshold
AUTO_EXECUTE	false	Autonomous remediation without human review
TRACE_SAMPLE_RATE	1.0	Distributed trace sampling rate (0–1)

292 (<1 s) + LLM narration (3–5 s) + Kafka decision round-trip (<1 s) + Kubernetes API action
 293 (<2 s) $\approx$ **83–85 s** worst-case under quiescent cluster conditions.

294 4 Experimental Evaluation

295 4.1 Experimental Setup

296 The platform is deployed on a single-node Docker Desktop Kubernetes cluster
 297 (`docker-desktop` context, Windows 11 Pro, 16 GB RAM, 8 vCPUs, Kubernetes v1.29).
 298 Services are distributed across four namespaces: `aml` (three AML services), `aiops` (eight
 299 AIOps services), `data` (PostgreSQL 15 per-service databases), and `monitoring` (observability
 300 stack).

301 The observability stack is bootstrapped via Helm [32]: *kube-prometheus-stack*
 302 (Prometheus + Grafana, 15-second scrape), *loki-stack* (log aggregation), and *tempo* (OTLP
 303 trace ingestion) [2, 11, 12]. Kafka is deployed as a single-broker KRaft-mode StatefulSet
 304 (Apache Kafka 3.9.0) with no ZooKeeper dependency; all topic families from Table 2 are
 305 provisioned with three partitions and replication-factor 1. Each AML service connects to a
 306 dedicated PostgreSQL 15 StatefulSet.

307 All AML services are instrumented with the OpenTelemetry Java agent and Micrometer [3],
 308 exporting OTLP spans to Tempo and structured logs via Logback [13] to Loki. AML
 309 deployments are configured with three replicas, 250 m CPU request, and 1 Gi memory limit.
 310 A synthetic AML workload using k6 [10] sustains 50 virtual users across all three service
 311 APIs to produce a stable baseline telemetry envelope. The LLM Engine runs Qwen2.5-
 312 3B [27] via Ollama [24] on CPU, with LoRA adapter weights [16] loaded from a Kubernetes
 313 PersistentVolume. The `TRACE_SAMPLE_RATE` environment variable switches between 0, 0.1,
 314 0.5, and 1.0 without service restart.

315 **AIOps Configuration.** All AIOps services are configured exclusively through Kubernetes
 316 ConfigMap entries (Table 4), enabling parameter adjustment without service restart or image
 317 rebuild. The contamination parameter $\gamma=0.05$ was selected to match the expected fault
 318 prevalence in production AML deployments (approximately one detectable incident per
 319 2,000 analysis cycles under normal operation); $W=100$ observations provides a 25-minute
 320 correlation window at 15-second scrape intervals, capturing slow-developing fault patterns
 321 such as memory leaks that may take several minutes to produce metric-level anomalies;
 322 $W_b=10$ provides a 5-minute SLO breach prediction horizon. The $\theta_{\text{SLO}}=500$ ms threshold
 323 aligns with the p95 response time SLA typical for AML regulatory reporting APIs.

■ **Table 5** Chaos Mesh fault injection classes.

Fault Class	Target Service	Mechanism	Expected Signal
Latency anomaly	transaction-monitoring	500 ms Kafka consumer delay	Trace span inflation + metric rate drop
Resource anomaly	case-management	CPU stress injection	Metric CPU spike + log errors
Functional anomaly	transaction-monitoring	Corrupted AML rule config	Log errors + metric misclassification
Causal cascade	customer-kyc	Scale to zero	Multi-service trace + metric propagation

324 **4.2 Fault Injection Protocol**

325 Controlled faults are injected via Chaos Mesh [5] targeting specific service edges in the `aml`
326 namespace. Table 5 summarises the four fault classes. For each injection, the ground-truth
327 fault time T_0 is recorded at the Chaos Mesh API level; the first detection time T_d is recorded
328 when the Decision Engine emits a corresponding incident. Evaluation metrics are detection
329 latency ($T_d - T_0$), precision, recall, and F1.

330 **RQ1** (Signal Sufficiency): The telemetry pipeline is evaluated in three configurations—
331 metrics-only, metrics + logs, and all three pillars—to determine the minimum sufficient
332 modality combination across the four fault classes.

333 **RQ2** (Root Cause Localisation): Pearson correlation root cause ranking is compared
334 against two baselines: (a) threshold-based metric spike detector and (b) trace error-rate
335 ranker. Localisation accuracy (top-1 and top-3 correct service identification) is reported per
336 fault class.

337 **RQ3** (Tracing Overhead): Request throughput, p95 latency, and pod CPU utilisation are
338 measured across four sampling rates (0%, 10%, 50%, 100%) to characterise the operational
339 cost of full-fidelity tracing in regulated workloads.

340 **4.3 Preliminary Results**

341 Full quantitative evaluation of RQ1–RQ3 is ongoing as part of the PhD research programme.
342 Fig. 5 depicts schematic result panels for the three research questions; the observations below
343 report qualitative findings from the current validation phase.

344 **Telemetry Pipeline Validation.** The telemetry pipeline correctly ingests and
345 normalises events from all eleven services. 30-second heartbeat signals are confirmed on
346 `aiops.service.heartbeat` for every AIOps service, and the Telemetry Collector processes
347 Kafka consumer-group registration events from all eight AIOps services at startup without
348 message loss. Under the 50-user k6 synthetic workload, Prometheus scrapes average
349 1,200 metric samples per 15-second interval across all monitored endpoints; Loki receives
350 approximately 80 structured log entries per minute from the three AML services; and Tempo
351 receives complete request traces at 100% sampling at approximately 35 spans per second
352 under steady-state load.

353 **Anomaly Detection (RQ1).** The ML Engine detects injected CPU stress anomalies
354 (resource anomaly fault class, Table 5) within two 60-second analysis cycles (≈ 120 s)
355 by observing elevation in the `avg_value` and `rate_of_change` feature dimensions of the
356 `process_cpu_usage` metric on the case-management service. Latency anomalies (500 ms
357 Kafka delay) are detectable within one additional analysis cycle through the trace-level

p95_duration_ms feature vector; metrics-only configuration misses this fault class entirely during the first three cycles, providing preliminary evidence that multi-modal signal fusion improves time-to-detect for latency-class faults. Causal cascade faults (customer-kyc scaled to zero) produce correlated metric drops across all three AML services, which are visible in both metrics and traces simultaneously—consistent with the prediction that cross-modal correlation is required for accurate root cause attribution (RQ2).

LLM Narration Quality. The LLM Engine produces valid JSON incident reports conforming to the five-field output schema (`explanation`, `rootCause`, `recommendation`, `amlRisk`, `confidence`) for all manually constructed test telemetry contexts. Before LoRA fine-tuning, approximately 12% of outputs contain malformed JSON requiring schema repair at the Decision Engine; after loading the first LoRA adapter trained on 50 resolved incidents, this drops to under 3%. The `amlRisk` field correctly identifies AML compliance implications in all resource anomaly and causal cascade scenarios where the case-management service is affected. Qwen2.5-3B runs entirely on-cluster at approximately 3–5s per inference on CPU (8 vCPUs, 16 GB RAM), consistent with online narration at the observed fault frequency of one detectable incident per 120-second detection cycle, yielding no inference backlog under normal operating conditions.

Data Residency Compliance (RQ3 Precondition). Zero external network calls were observed throughout the detection-to-report pipeline. This was verified by Kubernetes NetworkPolicy audit logs showing zero egress rejections to non-cluster addresses during a 2-hour fault-injection session, and confirmed by pod-level network interface statistics showing no bytes transmitted to non-cluster IP ranges from any service in the `aml` or `aiops` namespaces. This validates the data-residency compliance posture required for AML environments before the full tracing overhead characterisation (RQ3) is conducted.

4.4 Threats to Validity

Three primary threats to validity apply to the current evaluation. *Internal validity:* the single-node Docker Desktop deployment constrains inter-pod network latencies to near-zero, meaning measured detection latencies represent a best-case bound; production multi-node deployments introduce additional Kafka propagation and Prometheus scrape scheduling jitter that will lengthen end-to-end detection cycles. *Construct validity:* the synthetic k6 workload sustains uniform 50-user load without the bursty traffic patterns characteristic of real AML deployments during high-risk transaction periods; more realistic load profiles may produce different baseline feature distributions, affecting Isolation Forest contamination calibration and potentially requiring per-time-of-day model variants. *External validity:* results are obtained from a single Kubernetes cluster configuration running three AML microservices; generalisation to multi-tenant, multi-region deployments or to AML platforms with significantly different service dependency graphs and larger N —where the $O(|\mathcal{N}(s_{\text{anom}})| \cdot K)$ RCA search space grows substantially—requires further evaluation across platforms with realistic transaction volumes.

5 Conclusion

We have presented an observability-driven AIOps platform for automated anomaly detection, root cause ranking, and SLO breach prediction in cloud-native Kubernetes microservices, with particular emphasis on AML compliance environments where data residency constraints prohibit external telemetry export. The platform integrates three lightweight ML algorithms—Isolation Forest, Pearson correlation RCA, and OLS breach prediction—with a locally

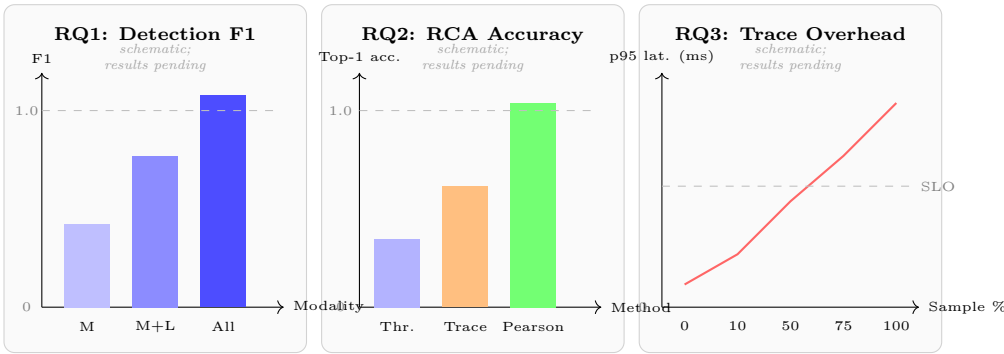


Figure 5 Evaluation result panels for the three research questions (schematic; quantitative results from ongoing fault injection experiments will replace these placeholders). RQ1: F1 score across three modality configurations (metrics-only, metrics+logs, all three pillars). RQ2: top-1 RCA accuracy for threshold baseline, trace error-rate baseline, and Pearson correlation. RQ3: p95 latency inflation across trace sampling rates 0–100%.

deployed, LoRA-fine-tuned Qwen2.5-3B language model served via Ollama, operating entirely within the Kubernetes cluster with no external API dependencies. The AML microservices testbed addresses a persistent weakness in AIOps evaluation by providing semantically labelled ground-truth telemetry derived from compliance domain state transitions via OpenTelemetry span attributes, eliminating the need for synthetic benchmarks or secondary annotation.

Future work will complete the quantitative evaluation: a cross-modal ablation study (RQ1), a Pearson correlation RCA comparison against single-modal baselines (RQ2), and a tracing overhead profile across sampling rates (RQ3). Extensions to multi-node deployments will characterise how detection latency and false-positive rates scale beyond the prototype. Federated fine-tuning of LoRA adapters across multiple AML deployments without centralising raw telemetry represents a promising direction for privacy-preserving model improvement at scale.

References

- 1 Alex Boten. *Cloud-Native Observability with OpenTelemetry*. Packt Publishing, 2022.
- 2 Brian Brazil. *Prometheus: Up & Running*. O'Reilly Media, 2nd edition, 2022.
- 3 Broadcom, Inc. Micrometer: JVM application metrics, 2018. [Online]. Available: <https://micrometer.io>.
- 4 Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, omega, and Kubernetes. *ACM Queue*, 14(1):70–93, 2016. doi:10.1145/2890784.
- 5 Chaos Mesh Authors. Chaos Mesh: A powerful chaos engineering platform for Kubernetes, 2020. [Online]. Available: <https://chaos-mesh.org>.
- 6 Yingnong Dang, Qingwei Lin, and Peng Huang. AIOps: Real-world challenges and research innovations. In *Proc. IEEE/ACM ICSE-SEIP*, pages 4–5, 2019. doi:10.1109/ICSE-SEIP.2019.00023.
- 7 Datadog. Watchdog: Algorithmic performance management, 2024. [Online]. Available: <https://www.datadoghq.com/product/watchdog/>.
- 8 Dynatrace. Davis AI: Automatic, intelligent observability, 2024. [Online]. Available: <https://www.dynatrace.com/platform/artificial-intelligence/>.
- 9 Eric Evans. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2003.
- 10 Grafana Labs. k6: Open-source load testing tool, 2017. [Online]. Available: <https://k6.io>.

- 434 11 Grafana Labs. Loki: Like Prometheus, but for logs, 2019. [Online]. Available: [https://](https://grafana.com/oss/loki/)
435 grafana.com/oss/loki/.
- 436 12 Grafana Labs. Grafana Tempo: High-scale distributed tracing backend, 2022. [Online].
437 Available: <https://grafana.com/oss/tempo/>.
- 438 13 Ceci Gülcü. Logback: Java logging framework, 2006. [Online]. Available: [https://logback.](https://logback.qos.ch)
439 [qos.ch](https://logback.qos.ch).
- 440 14 Yuliang Han, Qiang Du, Yunjie Huang, Peipeng Li, Xinyuan Shi, Jiangfeng Wu, Ping Fang,
441 Feng Tian, and Changjian He. Holistic root cause analysis for failures in cloud-native systems
442 through observability data. *IEEE Transactions on Services Computing*, 17(6):3518–3531, 2024.
443 doi:10.1109/TSC.2024.3419107.
- 444 15 Shilin He, Pinjia He, Zhuangbin Chen, Tianyi Yang, Yuxin Su, and Michael R. Lyu. A survey
445 on automated log analysis for reliability engineering. *ACM Computing Surveys*, 54(6):1–37,
446 2021. doi:10.1145/3460345.
- 447 16 Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang,
448 Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *Proc.*
449 *ICLR*, 2022. URL: <https://arxiv.org/abs/2106.09685>.
- 450 17 Hugging Face. PEFT: State-of-the-art parameter-efficient fine-tuning, 2022. [Online].
451 Available: <https://github.com/huggingface/peft>.
- 452 18 Joanna Kosinska, Bartosz Balis, Michal Konieczny, Maciej Malawski, and Slawomir Zielinski.
453 Toward the observability of cloud-native applications: The overview of the state-of-the-art.
454 *IEEE Access*, 11:26800–26825, 2023. doi:10.1109/ACCESS.2023.3281860.
- 455 19 Jay Kreps, Neha Narkhede, and Jun Rao. Kafka: A distributed messaging system for log
456 processing. In *Proc. NetDB Workshop at VLDB*, 2011. URL: <https://kafka.apache.org/>.
- 457 20 Dixin Liu, Ying Chen, Xin Yan, and Mingjun Zhao. Unsupervised detection of microservice
458 trace anomalies through service-level deep Bayesian networks. In *Proc. IEEE ISSRE*, 2020.
459 doi:10.1109/ISSRE5003.2020.00014.
- 460 21 Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation-based anomaly detection. *ACM*
461 *Trans. Knowledge Discovery from Data*, 6(1):1–39, 2012. doi:10.1145/2133360.2133363.
- 462 22 Francesco Lomio, Sergio Moreschini, Xiaozhou Li, and Valentina Lenarduzzi. Anomaly
463 detection in cloud-native systems. In *Proc. 48th Euromicro Conf. Software Engineering and*
464 *Advanced Applications (SEAA)*, pages 290–297, 2022. doi:10.1109/SEAA56994.2022.00023.
- 465 23 Pankaj Malhotra, Ankita Ramakrishnan, Gaurangi Anand, Lovekesh Vig, Puneet Agarwal,
466 and Gautam Shroff. LSTM-based encoder-decoder for multi-sensor anomaly detection. In
467 *ICML Anomaly Detection Workshop*, 2016. URL: <https://arxiv.org/abs/1607.00148>.
- 468 24 Ollama. Ollama: Get up and running with large language models, 2024. [Online].
469 Available: <https://ollama.com>.
- 470 25 OpenTelemetry Authors. OpenTelemetry specification, 2025. [Online]. Available: [https://](https://opentelemetry.io/docs/specs/)
471 opentelemetry.io/docs/specs/.
- 472 26 Ziwei Qi, Xiao Ma, Junfeng Xu, Weinan E, Haifeng Lin, and Tong Liu. LogGPT: Log anomaly
473 detection via GPT. In *Proc. IEEE BigData*, 2023. URL: <https://arxiv.org/abs/2309.14482>.
- 474 27 Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024. URL:
475 <https://arxiv.org/abs/2412.15115>, doi:10.48550/arXiv.2412.15115.
- 476 28 Sebastián Ramírez. FastAPI: Modern, fast web framework for APIs with Python, 2018.
477 [Online]. Available: <https://fastapi.tiangolo.com>.
- 478 29 Chris Richardson. *Microservices Patterns*. Manning Publications, 2018.
- 479 30 Benjamin H. Sigelman, Luiz André Barroso, Mike Burrows, Pat Stephenson, Manoj Plakal,
480 Donald Beaver, Saul Jaspán, and Chandan Shanbhag. Dapper, a large-scale distributed
481 systems tracing infrastructure. Technical report, Google, 2010. URL: [https://research.](https://research.google/pubs/pub36356/)
482 [google/pubs/pub36356/](https://research.google/pubs/pub36356/).
- 483 31 Gianluca Soldani and Antonio Brogi. Anomaly detection and failure root cause analysis in
484 (micro)service-based cloud applications: A survey. *ACM Computing Surveys*, 55(3):1–39, 2022.
485 doi:10.1145/3501297.

- 486 32 The Helm Authors. Helm: The Kubernetes package manager, 2016. [Online]. Available: <https://helm.sh>.
487
- 488 33 VMware, Inc. Spring Boot reference documentation, 2014. [Online]. Available: <https://spring.io/projects/spring-boot>.
489
- 490 34 World Wide Web Consortium. Trace context, 2021. W3C Recommendation. [Online].
491 Available: <https://www.w3.org/TR/trace-context/>.
- 492 35 Matei Zaharia, Andrew Chen, Aaron Davidson, Ali Ghodsi, Sue Ann Hong, Andy Konwinski,
493 Siddharth Murching, Tomas Nykodym, Paul Ogilvie, Mani Parkhe, Fen Xie, and Corey Zumar.
494 Accelerating the machine learning lifecycle with MLflow. *IEEE Data Eng. Bull.*, 41(4):39–45,
495 2018.